Algoritmos de Optimización — Problema 3: combinar cifras y operaciones

- Luis Chumpitaz Santivañez
- Everth Palomino Lanchipa

GitHub: https://github.com/epalominola/AlgoritmoOptimizacion/tree/main/proyectoTrabajoPractico

Enunciado

El problema consiste en analizar el siguiente problema y diseñar un algoritmo que lo resuelva.

Disponemos de las 9 cifras del 1 al 9 (excluimos el cero) y de los 4 signos básicos de las operaciones fundamentales: suma(+), resta(-), multiplicación(*) y división(/).

Debemos combinarlos alternativamente sin repetir ninguno de ellos para obtener una cantidad dada. Un ejemplo sería para obtener el 4: 4+2-6/3*1 = 4

Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?
- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo ?

Nota: Es posible usar la función de python “eval” para evaluar una expresión:

Desarrollo por fuerza bruta

La idea es construir **todas** las expresiones posibles alternando cifras y operadores, sin repetir ningún elemento, aquí se presenta directamente construida sobre `itertools.permutations`

```
import itertools


def es_entero(expresion):
    """Devuelve True si la expresión evaluada da como resultado un número entero."""
    valor = eval(expresion)
    return valor == int(valor)


def generar_expresiones(cifras=('1', '2', '3', '4', '5', '6', '7', '8', '9'),
                        operadores=('+', '-', '/', '*'),
                        k=5,
                        solo_enteros=True):
    """
    Construye todas las expresiones que alternan `k` cifras (sin repetir) con
    permutaciones de `operadores` (sin repetir), del tipo cifra-op-cifra-op-...-cifra.
    """
    permutaciones_cifras = itertools.permutations(cifras, k)
    permutaciones_operadores = list(itertools.permutations(operadores))

    expresiones = []

    for secuencia_cifras in permutaciones_cifras:
        for secuencia_operadores in permutaciones_operadores:
            partes = []
            for cifra, operador in zip(secuencia_cifras, secuencia_operadores):
                partes.append(cifra)
                partes.append(operador)
            partes.append(secuencia_cifras[-1])            # la última cifra no lleva operador detrás

            expresion = ''.join(partes)

            if not solo_enteros or es_entero(expresion):
                expresiones.append(expresion)

    return expresiones


expresiones = generar_expresiones()
print(f'Teniendo en cuenta que el resultado debe ser un entero, hay {len(expresiones)} combinaciones posibles.')
```

```
Teniendo en cuenta que el resultado debe ser un entero, hay 90000 combinaciones posibles.
```

Complejidad del algoritmo por fuerza bruta

El coste crece por dos vías independientes:

- los **operadores**: con n operadores distintos hay $n!$ formas de ordenarlos (permutación sin repetición);
- las **cifras**: eligiendo k cifras de un total de m, respetando el orden y sin repetir, hay $m!/(m-k)!$ formas posibles (variación sin repetición).

$$permutaciones(n) = n! \qquad variaciones(m,k) = \frac{m!}{(m-k)!}$$

de modo que la complejidad total del algoritmo es:

$$O\left(n! \cdot \frac{m!}{(m-k)!}\right)$$

Tamaño del espacio sin la restricción de ser enteros

Si no se exige que el resultado sea entero, el tamaño del espacio es exactamente $4! \cdot 9!/(9-5)! = 362\,880$. Se comprueba reutilizando la misma función con `solo_enteros=False`.

```
todas_las_combinaciones = generar_expresiones(solo_enteros=False)
print(f'Sin exigir que el resultado sea entero hay {len(todas_las_combinaciones)} combinaciones posibles.')
```

```
Sin exigir que el resultado sea entero hay 362880 combinaciones posibles.
```

Explorando el espacio de posibilidades: ¿cuál es el máximo y el mínimo?

Se comparan dos formas directas de encontrar el máximo y el mínimo del espacio de expresiones válidas (una tercera forma, basada en un diccionario, se desarrolla más adelante porque además sirve para el resto del análisis).

Método A — recorrido único: se recorre la lista de expresiones una sola vez, actualizando el máximo y el mínimo sobre la marcha.

```
def buscar_max_y_min(espacio_de_posibilidades):
    """Recorre el espacio de expresiones una sola vez y devuelve (mínimo, máximo)."""
    minimo = maximo = eval(espacio_de_posibilidades[0])

    for expresion in espacio_de_posibilidades[1:]:
        valor = eval(expresion)
        minimo = min(minimo, valor)
        maximo = max(maximo, valor)

    return minimo, maximo


buscar_max_y_min(expresiones)
```

```
(-69.0, 77.0)
```

Método B — evaluar todo y usar `min` / `max`: se evalúan todas las expresiones de una vez y se delega en las funciones nativas de Python.

```
def evaluar_espacio_de_posibilidades(espacio_de_posibilidades):
    """Devuelve la lista de resultados numéricos de cada expresión del espacio dado."""
    return [eval(expresion) for expresion in espacio_de_posibilidades]


valores = evaluar_espacio_de_posibilidades(expresiones)
print(min(valores))
print(max(valores))
```

```
-69.0
77.0
```

¿Es posible encontrar todos los enteros entre ese mínimo y ese máximo?

Sí. Se comprueba de dos maneras independientes y se verifica que coinciden:

- generando el rango completo de enteros entre el mínimo y el máximo;
- quedándose con el conjunto (`set`) de resultados realmente obtenidos y ordenándolo.

```
maximo, minimo = int(max(valores)), int(min(valores))
rango_completo = list(range(minimo, maximo + 1))

print(rango_completo)
print(len(rango_completo))
```

```
[-69, -68, -67, -66, -65, -64, -63, -62, -61, -60, -59, -58, -57, -56, -55, -54, -53, -52, -51, -50, -49, -48, -47, -46, -45, -4
147
```

```
resultados_obtenidos = sorted(set(valores))

print(resultados_obtenidos)
print(len(resultados_obtenidos))
```

```
[-69.0, -68.0, -67.0, -66.0, -65.0, -64.0, -63.0, -62.0, -61.0, -60.0, -59.0, -58.0, -57.0, -56.0, -55.0, -54.0, -53.0, -52.0, -
147
```

```
print(len(rango_completo) == len(resultados_obtenidos))   # confirma que se alcanzan todos los enteros del intervalo
```

```
True
```

Preguntas sobre el planteamiento del problema

¿Cuántas posibilidades hay sin tener en cuenta las restricciones? 362.880, calculadas en la sección 2 como $4! \cdot 9!/(9-5)!$.

¿Cuántas hay teniendo en cuenta la restricción de que el resultado sea entero? 90.000, calculadas en la sección 2.

¿Qué estructura de datos se adapta mejor al problema? Principalmente las **listas**: permiten modificar valores, cambiar de tamaño y acceder por índice, además de mantener el orden, que es esencial para construir la expresión cifra-operador-cifra-operador. También se usan tuplas y conjuntos (`set`) para casos puntuales, y más adelante un diccionario, que resulta clave para organizar los resultados.

¿Cuál es la función objetivo? Para construir el espacio de posibilidades, la función objetivo es la que determina si el resultado de una expresión es entero. Para la búsqueda de todos los enteros del intervalo no hay propiamente una función objetivo: el objetivo es reducir el espacio a explorar. Una vía posible sería atacar directamente las operaciones que pueden introducir decimales —la división, y su posible compensación con una multiplicación posterior— en lugar de evaluar la expresión completa y descartarla después.

¿Es un problema de maximización o de minimización? Ninguno de los dos en sentido estricto: es un **problema de búsqueda**. El objetivo es recorrer el espacio de posibilidades con un algoritmo que no resulte demasiado costoso computacionalmente, no optimizar una función objetivo continua.

Organizando los resultados: Diccionario de expresiones

Guardar los resultados en listas obliga a recorrerlas por completo cada vez que se necesita algo. Resulta mucho más cómodo construir un **diccionario** donde la llave es el resultado numérico de la expresión y el valor es la lista de expresiones que lo producen. Esto no reduce la complejidad del algoritmo de fuerza bruta (se sigue evaluando todo el espacio), pero facilita enormemente el análisis posterior: frecuencias, patrones, máximos, mínimos, etc.

```
def dct_espacio_de_posibilidades(todas_las_expresiones):
    """
    Agrupa las expresiones por su resultado: llave = resultado entero,
    valor = lista de expresiones que producen ese resultado.
    El diccionario se devuelve ordenado por llave ascendente.
    """
    agrupado = {}
    for expresion in todas_las_expresiones:
        resultado = int(eval(expresion))
        agrupado.setdefault(resultado, []).append(expresion)

    return dict(sorted(agrupado.items()))
```

```
dct_expresiones = dct_espacio_de_posibilidades(expresiones)
print(f'El máximo es {max(dct_expresiones)} y el mínimo {min(dct_expresiones)}')
```

```
El máximo es 77 y el mínimo -69
```

Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta.

Búsqueda aleatoria

A diferencia de la fuerza bruta, este algoritmo no recorre el espacio de posibilidades: genera directamente `n` expresiones aleatorias válidas (del tamaño que se le pida, 5 cifras y 4 operadores aquí). El número de operaciones depende únicamente del tamaño de la muestra solicitada, no de cuántas cifras u operadores haya disponibles:

$$complejidad = O(n)$$

siendo n el tamaño de la muestra.

```
import random


def evaluar_expresion(expresion):
    """Devuelve True si el resultado de la expresión es un número entero."""
    valor = eval(expresion)
    return valor == int(valor)


def generar_secuencia_operacional_aleatoria(cifras=['1', '2', '3', '4', '5', '6', '7', '8', '9'],
                                            operadores=['+', '-', '/', '*']):
    """Genera una única expresión aleatoria válida, o None si el resultado no es entero."""
    cifras_mezcladas = cifras.copy()
    operadores_mezclados = operadores.copy()
    random.shuffle(cifras_mezcladas)
    random.shuffle(operadores_mezclados)

    cifras_elegidas = cifras_mezcladas[:5]

    partes = []
    for cifra, operador in zip(cifras_elegidas, operadores_mezclados):
        partes.append(cifra)
        partes.append(operador)
    partes.append(cifras_elegidas[-1])

    expresion = ''.join(partes)
    return expresion if evaluar_expresion(expresion) else None


def muestra_aleatoria_de_secuencias_operacionales(tamano_muestra,
                                                  cifras=['1', '2', '3', '4', '5', '6', '7', '8', '9'],
                                                  operadores=['+', '-', '/', '*']):
    """Genera una muestra aleatoria de `tamano_muestra` expresiones válidas, agrupadas en diccionario."""
    muestra = []
    while len(muestra) < tamano_muestra:
        expresion = generar_secuencia_operacional_aleatoria(cifras, operadores)
        if expresion is not None:
            muestra.append(expresion)

    return dct_espacio_de_posibilidades(muestra)


def expresion_maxima(dct_muestra):
    """Devuelve (valor máximo, expresiones que lo alcanzan) dentro de una muestra."""
    maximo = max(dct_muestra)
    return maximo, dct_muestra[maximo]


def expresion_minima(dct_muestra):
    """Devuelve (valor mínimo, expresiones que lo alcanzan) dentro de una muestra."""
    minimo = min(dct_muestra)
    return minimo, dct_muestra[minimo]
```

```
muestra = muestra_aleatoria_de_secuencias_operacionales(1000)
print(f'Máximo de la muestra aleatoria: {expresion_maxima(muestra)}')
print(f'Mínimo de la muestra aleatoria: {expresion_minima(muestra)}')
```

```
Máximo de la muestra aleatoria: (76, ['9*8+6-4/2'])
Mínimo de la muestra aleatoria: (-65, ['5-8*9/1+2'])
```

Búsqueda local

Se parte de la búsqueda aleatoria anterior y, a partir del mejor máximo y mínimo encontrados, se exploran `p` "vecinos" de cada uno (expresiones obtenidas intercambiando dos posiciones del mismo tipo —dos cifras o dos operadores— para que la expresión siga teniendo sentido matemático).

Pasos del algoritmo:

1. Genera una muestra aleatoria de `n` expresiones.
2. Se queda con una expresión que alcanza el máximo y otra que alcanza el mínimo.
3. Para cada una, explora `p` vecinos.

$$complejidad = O(2 \cdot p \cdot n)$$

Y si se repite el proceso `x` veces (**búsqueda local con reinicio múltiple**, más adelante):

$$complejidad = O(2 \cdot p \cdot n \cdot x)$$

```
def generar_secuencias_operacionales_vecinas(expresion, limite_iteraciones=100):
    """
    Genera expresiones "vecinas" intercambiando dos posiciones de la expresión original.
    Los índices a intercambiar deben ser ambos pares o ambos impares, para que el
    intercambio siga alternando cifra-operador correctamente (y no produzca, por
    ejemplo, '3+*12-5/1').
    """
    indices = list(range(len(expresion)))
    expresion = list(expresion)
    vecinas = []

    for _ in range(limite_iteraciones):
        while True:
            i, j = random.sample(indices, 2)
            if (i % 2 == 0) == (j % 2 == 0):      # ambos pares o ambos impares
                break

        i_min, i_max = min(i, j), max(i, j)
        vecina = (expresion[:i_min] + [expresion[i_max]]
                  + expresion[i_min + 1:i_max] + [expresion[i_min]]
                  + expresion[i_max + 1:])
        vecina = ''.join(vecina)

        if evaluar_expresion(vecina):
            vecinas.append(vecina)

    return dct_espacio_de_posibilidades(vecinas)


def busqueda_local(cifras=['1', '2', '3', '4', '5', '6', '7', '8', '9'],
                   operadores=['+', '-', '/', '*']):
    """
    Genera una muestra aleatoria, obtiene su máximo y mínimo, explora vecinos de
    ambos y se queda con el mejor resultado entre la muestra aleatoria y sus vecinos.
    Imprime los resultados de ambas fases; no devuelve nada.
    """
    muestra = muestra_aleatoria_de_secuencias_operacionales(1000, cifras=cifras, operadores=operadores)
    maximo_muestra = expresion_maxima(muestra)
    minimo_muestra = expresion_minima(muestra)
    print(f'Máximo de la búsqueda aleatoria: {maximo_muestra[0]}, expresiones: {maximo_muestra[1]}')
    print(f'Mínimo de la búsqueda aleatoria: {minimo_muestra[0]}, expresiones: {minimo_muestra[1]}\n')

    vecinos_del_maximo = generar_secuencias_operacionales_vecinas(maximo_muestra[1][0])
    vecinos_del_minimo = generar_secuencias_operacionales_vecinas(minimo_muestra[1][0])
    mejor_vecino_maximo = expresion_maxima(vecinos_del_maximo)
    mejor_vecino_minimo = expresion_minima(vecinos_del_minimo)

    maximo_final = mejor_vecino_maximo if mejor_vecino_maximo[0] > maximo_muestra[0] else maximo_muestra
    minimo_final = mejor_vecino_minimo if mejor_vecino_minimo[0] < minimo_muestra[0] else minimo_muestra

    print(f'Máximo de la búsqueda local: {maximo_final[0]}, expresiones: {maximo_final[1]}')
    print(f'Mínimo de la búsqueda local: {minimo_final[0]}, expresiones: {minimo_final[1]}')
```

```
busqueda_local()
```

```
Máximo de la búsqueda aleatoria: 76, expresiones: ['6+8*9-2/1']
Mínimo de la búsqueda aleatoria: -67, expresiones: ['3-8/1*9+2']

Máximo de la búsqueda local: 76, expresiones: ['6+8*9-2/1']
Mínimo de la búsqueda local: -67, expresiones: ['3-8/1*9+2']
```

Los resultados no mejoran de forma apreciable respecto a la búsqueda aleatoria simple: generar vecinos no aporta demasiado en este caso. Se corrobora repitiendo el experimento varias veces (**búsqueda local con reinicio múltiple**, 5 repeticiones).

```
for repeticion in range(5):
    busqueda_local()
    print('\n' + '\t\t\t' + '#' * 100 + '\n')
```

```
Máximo de la búsqueda aleatoria: 77, expresiones: ['9*8-4/2+7']
Mínimo de la búsqueda aleatoria: -66, expresiones: ['4-9*8/1+2']

Máximo de la búsqueda local: 77, expresiones: ['9*8-4/2+7']
Mínimo de la búsqueda local: -66, expresiones: ['4-9*8/1+2']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 76, expresiones: ['8/1*9+7-3']
Mínimo de la búsqueda aleatoria: -66, expresiones: ['4/1-9*8+2']

Máximo de la búsqueda local: 76, expresiones: ['8/1*9+7-3']
Mínimo de la búsqueda local: -69, expresiones: ['4/2-9*8+1', '4/2-9*8+1', '4/2-9*8+1', '4/2-9*8+1', '4/2-9*8+1', '4/2-9*8+1',

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 75, expresiones: ['8/1*9-3+6', '9*8+5-4/2']
Mínimo de la búsqueda aleatoria: -66, expresiones: ['4-9/1*8+2']

Máximo de la búsqueda local: 75, expresiones: ['8/1*9-3+6', '9*8+5-4/2']
Mínimo de la búsqueda local: -66, expresiones: ['4-9/1*8+2']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 77, expresiones: ['9*8-6/3+7']
Mínimo de la búsqueda aleatoria: -63, expresiones: ['6+3-9/1*8']

Máximo de la búsqueda local: 77, expresiones: ['9*8-6/3+7']
Mínimo de la búsqueda local: -63, expresiones: ['6+3-9/1*8']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 76, expresiones: ['7+9*8/1-3', '8*9/1+6-2']
Mínimo de la búsqueda aleatoria: -67, expresiones: ['4/2-8*9+3']

Máximo de la búsqueda local: 76, expresiones: ['7+9*8/1-3', '8*9/1+6-2']
Mínimo de la búsqueda local: -67, expresiones: ['4/2-8*9+3']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 76, expresiones: ['6+8*9-4/2', '6-2+8/1*9']
Mínimo de la búsqueda aleatoria: -69, expresiones: ['6/3+1-9*8']

Máximo de la búsqueda local: 76, expresiones: ['6+8*9-4/2', '6-2+8/1*9']
Mínimo de la búsqueda local: -69, expresiones: ['6/3+1-9*8']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 75, expresiones: ['9*8+6/1-3', '5/1-2+8*9']
Mínimo de la búsqueda aleatoria: -67, expresiones: ['3-9*8+2/1']

Máximo de la búsqueda local: 75, expresiones: ['9*8+6/1-3', '5/1-2+8*9']
Mínimo de la búsqueda local: -67, expresiones: ['3-9*8+2/1']

                        ####################################################################################################

Máximo de la búsqueda aleatoria: 76, expresiones: ['9*8+6-4/2']
```

Conclusiones

La búsqueda local no mejora de forma significativa los resultados de la búsqueda aleatoria simple, al menos para este tamaño de expresiones. Además, tal como están escritas, las funciones de este trabajo no separan correctamente cifras de dos o más dígitos, por lo

que no generalizan bien si aumenta el tamaño de las entradas (por ejemplo, al mezclar números de una y dos cifras). Por ello se da el trabajo por concluido en este punto.

En resumen, quedan implementadas la siguiente alternativa a la fuerza bruta:

- dos **heurísticas** (búsqueda aleatoria y búsqueda local con reinicio múltiple), sencillas mas con buenos resultados en la práctica.

Ambas reducen notablemente el espacio de posibilidades a explorar respecto a la fuerza bruta original. Aun así, el trabajo tiene bastante margen de mejora.

```
Comienza a programar o generar con IA.
```